

```
In [ ]: print("hello raja")
        print("hello asrar")
        print("hello danish")
```

```
In [ ]: if you want to run a piece of code multiple times

        if you want to use a piece of code multiple times

        if you call a piece of code multiple times

        functions
```

```
In [1]: num1=10
        num2=20
        add=num1+num2
        print("the addition of {} and {} is {}".format(num1,num2,add))
```

the addition of 10 and 20 is 30

```
In [ ]: print
        input
        import
        eval
        if
        else
        elif
        try
        except
        def
        type
```

```
In [ ]: def <function_name>():
        #####
        task
        #####
```

1.Function with out arguments

```
In [2]: def addition():
        num1=10
        num2=20
        print(num1+num2)
```

- if you run the function creation code it will not give the any output
- if you want to run that function , you need to call the function
- will not get any error and any output untill unless we call the function

```
In [6]: addition()
```

```
30
```

```
In [9]: def greet():  
        print('hai')  
        print('hello')
```

```
greet()
```

```
hai  
hello
```

```
In [8]: greet()
```

```
hai  
hello
```

```
In [10]: greet()
```

```
hai  
hello
```

```
In [11]: print(1)  
         print(2)  
         greet()  
         print('done')
```

```
1  
2  
hai  
hello  
done
```

```
In [12]: def addition1():  
        num1=10  
        num2=20  
        print(num1+num2)
```

```
In [13]: addition1()
```

```
-----  
-  
NameError                                Traceback (most recent call las  
t)  
Cell In[13], line 1  
----> 1 addition1()  
  
Cell In[12], line 4, in addition1()  
      2 num1=10  
      3 num2=20  
----> 4 print(num1+num2)  
  
NameError: name 'num11' is not defined
```

```
In [ ]: # Create a function name with average
# ask the user enter three numbers
# n1=eval(input())
# n2=eval(input())
# n3=eval(input())
# average= (n1+n2+n3)/3
# print(average)
```

```
In [ ]: - Function is like a wrapper

- we develop a normal code

- But this code will execute by a function call
```

```
In [3]: n1=20
n2=30
n3=40
add=n1+n2+n3
avg=round(add/3,2)
print("The average of {}, {} and {} is {}".format(n1,n2,n3,avg))
print(f"The average of {n1}, {n2} and {n3} is {avg}")
```

The average of 20, 30 and 40 is 30.0
The average of 20, 30 and 40 is 30.0

```
In [12]: def average():
n1=20
n2=30
n3=40
add=n1+n2+n3
avg=round(add/3,2)
print("The average of {}, {} and {} is {}".format(n1,n2,n3,avg))
print(f"The average of {n1}, {n2} and {n3} is {avg}")

# function is ready
```

```
In [13]: average()
# they are not using print statements
# they are feeling tht i'm not getting output
# their function name inside some variable names alos mentioning same name
# make sure that function name and variable names should be different
```

The average of 20, 30 and 40 is 30.0
The average of 20, 30 and 40 is 30.0

```
In [15]: # if you want to implment try-exception
# implement inside the function call
# try-except both should be align
def average():
    try:
        n1=20
        n2=30
        n3=40
        add=n1+n2+n3
        avg=round(add/0,2)
        print("The average of {}, {} and {} is {}".format(n1,n2,n3,avg))
        print(f"The average of {n1}, {n2} and {n3} is {avg}")

    except Exception as e:
        print(e)

average()
```

division by zero

```
In [16]: def average1():
    try:
        n1=eval(input("enter the number1:"))
        n2=eval(input("enter the number2:"))
        n3=eval(input("enter the number3:"))
        add=n1+n2+n3
        avg=round(add/3,2)
        print("The average of {}, {} and {} is {}".format(n1,n2,n3,avg))
        print(f"The average of {n1}, {n2} and {n3} is {avg}")

    except Exception as e:
        print(e)

average1()
```

```
enter the number1:30
enter the number2:40
enter the number3:50
The average of 30, 40 and 50 is 40.0
The average of 30, 40 and 50 is 40.0
```

```
In [19]: # WAP create the average function by taking values randomly
import random
def average2():
    try:
        n1=random.randint(1,20)
        n2=random.randint(1,20)
        n3=random.randint(1,20)
        add=n1+n2+n3
        avg=round(add/3,2)
        print("The average of {}, {} and {} is {}".format(n1,n2,n3,avg))
        print(f"The average of {n1}, {n2} and {n3} is {avg}")

    except Exception as e:
        print(e)

average2()
```

The average of 14, 7 and 18 is 13.0

The average of 14, 7 and 18 is 13.0

```
In [ ]: n1=30
n1=eval(input())
n1=random.randint(1,20)

#####
n1=20
n2=30
n3=40
add=n1+n2+n3
avg=round(add/3,2)
#####
n1=eval(input("enter the number1:"))
n2=eval(input("enter the number2:"))
n3=eval(input("enter the number3:"))
add=n1+n2+n3
avg=round(add/3,2)
#####
n1=random.randint(1,20)
n2=random.randint(1,20)
n3=random.randint(1,20)
add=n1+n2+n3
avg=round(add/3,2)
```

```
In [25]: import random
def average1():
    try:
        n1=random.randint(1,20)
        n2=random.randint(1,20)
        n3=random.randint(1,20)
        add=n1+n2+n3
        avg=(add/0)
        print('the average of {}, {}, {} is {}'.format(n1,n2,n3,avg))
    except Exception as e:
        print('Exception {}'.format(e))

average1()
```

Exception division by zero

```
In [ ]: # WAP ask the user enter bill amount
#       ask the user how much tip percentage you want to provide
#       calculate the total bil
#       create a function and call it
```

```
In [27]: # Normal code
bill=eval(input("enter the bill:"))
tip_per=eval((input("enter the tip in %:")))
tip_amount=bill*tip_per/100
total_bill=bill+tip_amount
print("The total bill is:",total_bill)
```

```
enter the bill:1000
enter the tip in %:10
The total bill is: 1100.0
```

```
In [29]: def resto():
    bill=eval(input("enter the bill:"))
    tip_per=eval((input("enter the tip in %:")))
    tip_amount=bill*tip_per/100
    total_bill=bill+tip_amount
    print("The total bill is:",total_bill)

resto()
```

```
enter the bill:2000
enter the tip in %:10
The total bill is: 2200.0
```

```
In [31]: def resto():
    try:
        bill=eval(input("enter the bill:"))
        tip_per=eval((input("enter the tip in %:")))
        tip_amount=bill*tip_per/100
        total_bill=bill+tip_amount
        print("The total bill is:",total_bill)

    except Exception as e:
        print(e)

resto()
```

```
enter the bill:2000
enter the tip in %:20
The total bill is: 2400.0
```

```
In [32]: # wap ask the user enter a number find wether it is an even or odd
# take numbers randomly also

# Normal code
# step-1: num=eval(input("enter "))
# step-2: if <condition>:
# step-3:     print("even")
# step-4: else:
# step-5:     print("odd")

print(f"{number} is even")
```

number is even

```
In [36]: num=eval(input("enter a number:"))
if num%2==0:
    print("{} is even".format(num))
    print(f"{num} is even")
    print("even number:",num)
else:
    print("{} is odd".format(num))
    print(f"{num} is odd")
    print("odd number:",num)
```

enter a number:11
11 is odd
11 is odd
odd number: 11

```
In [37]: # Dont equal function name, and variable name
def even_odd():
    num=eval(input("enter a number:"))
    if num%2==0:
        print("{} is even".format(num))
        print(f"{num} is even")
        print("even number:",num)
    else:
        print("{} is odd".format(num))
        print(f"{num} is odd")
        print("odd number:",num)
```

even_odd()

enter a number:23
23 is odd
23 is odd
odd number: 23

```
In [42]: def even_odd1():
    start=eval(input("provide the start value:"))
    last=eval(input("provide the last value"))
    num=random.randint(start,last)
    if num%2==0:
        print("{} is even".format(num))
        print(f"{num} is even")
        print("even number:",num)
    else:
        print("{} is odd".format(num))
        print(f"{num} is odd")
        print("odd number:",num)

even_odd1()
```

```
provide the start value:300
provide the last value:400
335 is odd
335 is odd
odd number: 335
```

```
In [ ]: add()
        average()
        resto()
        even_odd()

# every function has brackets
# if you pass anything inside brackets we called arguments or parameters
```

```
In [ ]: im not provided anything inside my function

**Function with out arguements**
```

2.Function with arguments or parameters

- In your original code how many variable are there
- In your original code how many input variables are there
- In your original code how many output variable are there
- Output variable create by using input variable
- whenever you want create function with argument only concentrate input variable
- provide the values while calling the function

```
In [43]: def addition():
          num1=10
          num2=20
          add=num1+num2
          print(add)

          addition()

# q1) How many 3: num1 num2 add
# q2) how many i/p: num1 and num2
# q3) how many output: add
```

30

```
In [47]: def addition(num1,num2):
          add=num1+num2
          print(add)

          addition(10,20)
```

30

```
In [ ]: # Create a function with arguments for average program
```

```
In [49]: def average1(n1,n2,n3):
          try:
              add=n1+n2+n3
              avg=round(add/3,2)
              print("The average of {}, {} and {} is {}".format(n1,n2,n3,avg))
              print(f"The average of {n1}, {n2} and {n3} is {avg}")

          except Exception as e:
              print(e)

          average1(10,20,30)

#q1) 5 i(3) + o(2)
```

The average of 10, 20 and 30 is 20.0

The average of 10, 20 and 30 is 20.0

```
In [ ]: # Bill payment
# Even_odd

# whatsapp group

# s
```

```
In [7]: print("I imported random package")
import random
print("I created a average function")
def average1():
    try:
        print("Im taking value berween 1 and 20")
        n1=random.randint(1,20)
        n2=random.randint(1,20)
        n3=random.randint(1,20)
        add=n1+n2+n3
        avg=add/3
        print(f"The average of {n1}, {n2} and {n3} is {avg}")

    except Exception as e:
        print(e)

print("Im doing average")
average1()
print("I completed average")
```

```
I imported random package
I created a average function
Im doing average
Im taking value berween 1 and 20
The average of 3, 13 and 13 is 9.666666666666666
I completed average
```

```
In [2]: round(2.6666,2)
```

```
Out[2]: 2.67
```

```
In [ ]: # The above function is function with out argument
```

```
In [10]: # Function with arguments
def average1(n1,n2,n3):
    try:
        print("n1:",n1)
        print("n2:",n2)
        print("n3:",n3)
        add=n1+n2+n3
        avg=round(add/3,2)
        print("The average of {}, {} and {} is {}".format(n1,n2,n3,avg))
        print(f"The average of {n1}, {n2} and {n3} is {avg}")

    except Exception as e:
        print(e)

average1(100,300,200)

#n1,n2,n3,add,avg
# input: n1,n2,n3
# output: add,avg
```

```
n1: 100
n2: 300
n3: 200
The average of 100, 300 and 200 is 200.0
The average of 100, 300 and 200 is 200.0
```

Function with Default arguments

```
In [12]: def average1(n1,n2,n3=300):
    try:
        print("n1:",n1)
        print("n2:",n2)
        print("n3:",n3)
        add=n1+n2+n3
        avg=round(add/3,2)
        print("The average of {}, {} and {} is {}".format(n1,n2,n3,avg))
        print(f"The average of {n1}, {n2} and {n3} is {avg}")

    except Exception as e:
        print(e)

average1(100,200)
```

```
n1: 100
n2: 200
n3: 300
The average of 100, 200 and 300 is 200.0
The average of 100, 200 and 300 is 200.0
```

```
In [ ]: # WAP ask the user enter income
# keep tax as default argument = 10
# print how much tax need to pay
# create a function with arguments and with out arguments
# print how much tax need to pay

# tax = default parameter
# income ====> with out argument    tax_cal()
# income ==== > with argument      tax_cal(income,tax=10)
```

```
In [16]: # Case-1: with out argument
def tax_pay1():
    income=eval(input("enter income:"))
    tax_per=eval(input("enter tax in %:"))
    tax_pay= income*tax_per/100
    print("Total tax pay is:",tax_pay)
```

```
tax_pay1()
```

```
# how many total variable :3
# income,tax_per,tax_pay
# input: income and tax_per
# output: tax_pay
```

```
enter income:10000
enter tax in %:10
Total tax pay is: 1000.0
```

```
In [17]: def tax_pay2(income,tax_per):  
         tax_pay= income*tax_per/100  
         print("Total tax pay is:",tax_pay)  
  
         tax_pay2(10000,10)
```

Total tax pay is: 1000.0

```
In [18]: # Tax_per ===== Default  
         # income ===== with argument  
         def tax_pay3(income,tax_per=10):  
             tax_pay= income*tax_per/100  
             print("Total tax pay is:",tax_pay)  
  
         tax_pay3(10000)
```

Total tax pay is: 1000.0

```
In [19]: # tax_per ===== Default param , income ===== with out argument  
         def tax_pay4(tax_per=10):  
             income=10000  
             tax_pay= income*tax_per/100  
             print("Total tax pay is:",tax_pay)  
  
         tax_pay4()
```

Total tax pay is: 1000.0

```
In [ ]: #####with out argument#####
def tax_pay1():
    income=eval(input("enter income:"))
    tax_per=eval(input("enter tax in %:"))
    tax_pay= income*tax_per/100
    print("Total tax pay is:",tax_pay)

tax_pay1()

#####with argument#####
def tax_pay2(income,tax_per):
    tax_pay= income*tax_per/100
    print("Total tax pay is:",tax_pay)

tax_pay2(10000,10)

#####default argument, income as with argument##
def tax_pay3(income,tax_per=10):
    tax_pay= income*tax_per/100
    print("Total tax pay is:",tax_pay)

tax_pay3(10000)

#####default argument, income as with out argument
def tax_pay4(tax_per=10):
    income=10000
    tax_pay= income*tax_per/100
    print("Total tax pay is:",tax_pay)

tax_pay4()
```

```
In [22]: # WAP ask the user bill amount and tip
# calculate total bill
# implement this program in all 4 ways like above

#####with out argument#####
def bill_pay1():
    bill=eval(input("enter bill:"))
    tip_per=eval(input("enter tip in %:"))
    total_bill_pay= bill+bill*tip_per/100
    print("Total bill pay is:",total_bill_pay)

#####with argument#####
def bill_pay2(bill,tip_per):
    total_bill_pay= bill+bill*tip_per/100
    print("Total bill pay is:",total_bill_pay)

#####default argument, bill as with argument###
def bill_pay3(bill,tip_per=10):
    total_bill_pay= bill+bill*tip_per/100
    print("Total bill pay is:",total_bill_pay)

#####default argument, bill as with out argument##
def bill_pay4(tip_per=10):
    bill=10000
    total_bill_pay= bill+bill*tip_per/100
    print("Total bill pay is:",total_bill_pay)

print("Calling first function")
bill_pay1()
print("Calling second function")
bill_pay2(10000,10)
print("Calling third function")
bill_pay3(10000)
print("Calling forth function")
bill_pay4()
```

```
Calling first function
enter bill:1000
enter tip in %:10
Total bill pay is: 1100.0
Calling second function
Total bill pay is: 11000.0
Calling third function
Total bill pay is: 11000.0
Calling forth function
Total bill pay is: 11000.0
```

```
In [23]: def bill_pay5(bill=1000,tip_per):
          total_bill_pay= bill+bill*tip_per/100
          print("Total bill pay is:",total_bill_pay)

          bill_pay5(10)

          # non default argumet= tip_per
          # dfault arg= bill

          # tip per follows bill
```

Cell In[23], line 1

```
def bill_pay5(bill=1000,tip_per):
    ^
```

SyntaxError: non-default argument follows default argument

Default arguments inside function should be at last

```
In [ ]: avg(n1,n2,n3=100) # No error
        avg(n1,n2=100,n3) # error
        avg(n1=100,n2,n3) # error
        avg(n1,n2=100,n3=100) # no error
        avg(n1=100,n2=100,n3) # error
        avg(n1=100,n2,n3=100) # error
        avg(n1=100,n2=100,n3=100) # no error
```

```
In [25]: def bill_pay5(bill,tip_per=10):
          #####
          print("bill:",bill)
          print("tip_per:",tip_per)
          total_bill_pay= bill+bill*tip_per/100
          print("Total bill pay is:",total_bill_pay)

          bill_pay5(1000,200)

          # what is the value tip_per
```

```
bill: 1000
tip_per: 200
Total bill pay is: 3000.0
```

```
In [26]: def bill_pay6(bill,tip_per=10):
          #####
          print("bill:",bill)
          tip_per=500
          bill=2000
          print("bill 2nd:",bill)
          print("tip_per:",tip_per)
          total_bill_pay= bill+bill*tip_per/100
          print("Total bill pay is:",total_bill_pay)
```

```
bill_pay6(1000,200)
```

```
# while you creating function ===== > 10
# while you call the function ===== > 200
# while function running ===== > 500
```

```
bill: 1000
tip_per: 500
Total bill pay is: 6000.0
```

- Basic functions
- Function with out arguments
- Function with arguments
- Function with default arguments

```
In [1]: import random
```

```
In [3]: random.randint(10,20)
```

```
Out[3]: 16
```

```
In [6]: complex(10,20)
```

```
Out[6]: (10+20j)
```

- while creating function what are the values
- while you call the function, what are the provided values
- while function running, are you providing any updated values

```
In [11]: def greet(name='Python'):
          name="Nagaraju"
          print(f"hello {name}")
```

```
greet('Greeshma')
```

```
# is this code which type
# name='greeshma'
```

```
hello Nagaraju
```

Return

```
In [12]: def tax_pay1():
            income=eval(input("enter income:"))
            tax_per=eval(input("enter tax in %:"))
            tax_pay= income*tax_per/100
            print("Total tax pay is:",tax_pay)

            tax_pay1()

            # in this above function the output variable is tax_pay
            # I want to use this tax_pay in future code
            # that means out of that specific function

            enter income:10000
            enter tax in %:10
            Total tax pay is: 1000.0
```

```
In [13]: tax_pay
            # we can use this tax_pay variable outside the function
            # when the function returns
```

```
-----
-
NameError                                Traceback (most recent call last)
Cell In[13], line 1
----> 1 tax_pay

NameError: name 'tax_pay' is not defined
```

```
In [22]: def tax_pay2():
            income=eval(input("enter income:"))
            tax_per=eval(input("enter tax in %:"))
            tax_pay= income*tax_per/100
            print("tax_pay:",tax_pay)
            return(tax_pay)
```

```
In [23]: tax_pay=tax_pay2()    # this function returns one values

            enter income:10000
            enter tax in %:10
            tax_pay: 1000.0
```

```
In [24]: tax_pay
```

```
Out[24]: 1000.0
```

```
In [ ]: Use I understood but why we need to return the value
            1000 lines of code
            some where some point
            some output of the function , you want to use in future lines of code
            that means outside the function
            if you dont apply return you cant use
```

```
In [26]: def add(a,b):
          c=a+b
          return(c)

          summ=add(3,5)
```

```
In [32]: def add(a,b):
          # a=3
          c=a+b
          # c=8
          return(a,c)

          a,c=add(3,5)  # a,c=3,8
          print(a) # int
          print(c) # int

          # Q1) how many values function returns
          # Number variables in function returns is equal to
          # number variables to store the values

          3
          8
```

```
In [28]: a=10,6
          a
```

```
Out[28]: (10, 6)
```

```
In [30]: a,b=10,6
          print("a",a)
          print("b",b)

          a 10
          b 6
```

```
In [34]: # WAP on bill payment
          # ask the user enter bill
          # ask the user how much tip you want to provide
          # return bill as well as tip amount

          def bill_pay():
              bill=eval(input("enter the bill:"))
              tip_per=eval(input("enter the tip in per:"))
              tip_amount=bill*tip_per/100
              total_bill=bill+tip_amount
              return(total_bill,tip_amount)

          total_bill,tip_amount=bill_pay()  # 1100,100

          enter the bill:1000
          enter the tip in per:10
```

```
In [35]: print(total_bill)
          print(tip_amount)

          1100.0
          100.0
```

```
In [38]: def bill_pay():
          bill=eval(input("enter the bill:"))
          tip_per=eval(input("enter the tip in per:"))
          tip_amount=bill*tip_per/100
          total_bill=bill+tip_amount
          return(total_bill,tip_amount,tip_per)

          total_bill,tip_amount,tip_per=bill_pay()
```

```
enter the bill:1000
enter the tip in per:10
```

```
In [39]: print(total_bill)
          print(tip_amount)
          print(tip_per)
```

```
1100.0
100.0
10
```

```
In [ ]: # How you are providing values
```

- Im providing values inside the function
 - Local variables
- Im providing values outside the function
 - **global** variables

```
In [46]: # How you are providing values
def bill_pay():
    bill1=1000
    tip_per=10
    tip_amount=bill1*tip_per/100
    total_bill=bill1+tip_amount
    return(total_bill,tip_amount)

total_bill,tip_amount=bill_pay()
print(total_bill)
print(tip_amount)
print(bill1)
```

```
1100.0
100.0
```

```
-----
-
NameError                                Traceback (most recent call last)
Cell In[46], line 12
      10 print(total_bill)
      11 print(tip_amount)
----> 12 print(bill1)
```

```
NameError: name 'bill1' is not defined
```

```
In [44]: bill=2000
tip_per=20

def bill_pay1():
    tip_amount=bill*tip_per/100
    total_bill=bill+tip_amount
    return(total_bill,tip_amount)

total_bill,tip_amount=bill_pay1()
print(total_bill)
print(tip_amount)

print(bill*tip_per)
```

```
2400.0
400.0
40000
```

```
In [43]: bill=2000
tip_per=20
#####
print(bill*tip_per)

40000
```

```
In [ ]: # How you are providing values
def bill_pay():
    bill1=1000
    tip_per=10
    tip_amount=bill1*tip_per/100
    total_bill=bill1+tip_amount
    return(total_bill,tip_amount)

total_bill,tip_amount=bill_pay()
print(total_bill)
print(tip_amount)
print(bill1)

bill=2000
tip_per=20

def bill_pay1():
    tip_amount=bill*tip_per/100
    total_bill=bill+tip_amount
    return(total_bill,tip_amount)

total_bill,tip_amount=bill_pay1()
print(total_bill)
print(tip_amount)

print(bill*tip_per)
```

```
In [49]: def bill_pay1():
    tip_amount=bill*tip_per/100
    total_bill=bill+tip_amount
    print("tip_amount:",tip_amount)
    print("total_bill:",total_bill)

    bill=3000
    tip_per=30

    bill_pay1()
```

```
tip_amount: 900.0
total_bill: 3900.0
```

```
In [51]: def bill_pay1():
    tip_amount=bill1*tip_per1/100
    total_bill=bill1+tip_amount
    print("tip_amount:",tip_amount)
    print("total_bill:",total_bill)

    bill_pay1()

    bill1=3000
    tip_per1=30
```

```
-----
-
NameError                                Traceback (most recent call las
t)
Cell In[51], line 8
      4     print("tip_amount:",tip_amount)
      5     print("total_bill:",total_bill)
----> 8 bill_pay1()
      10 bill1=3000
      11 tip_per1=30

Cell In[51], line 2, in bill_pay1()
      1 def bill_pay1():
----> 2     tip_amount=bill1*tip_per1/100
      3     total_bill=bill1+tip_amount
      4     print("tip_amount:",tip_amount)

NameError: name 'bill1' is not defined
```

```
In [53]: name='rakesh'   # 1

def greet(name='Alex'):
    name="Sahil"
    print(f"hello {name}")

name='Sanjay'   # after line 1
greet('python')

hello Sahil
```

```
In [54]: name # sahil
```

```
Out[54]: 'Sanjay'
```

```
In [55]: name='sanjay'  
name='mahaboob'  
name
```

```
Out[55]: 'mahaboob'
```

```
In [56]: name='rakesh'  
  
def greet(name='Alex'):  
    print(f"hello {name}")  
  
name='Sanjay'  
greet('python')  
  
# first name = 'rakesh'  
# second name='alex' ===== > no use  
# Thirs name='Sanjay'  
# Fourth name= 'python'
```

```
hello python
```

```
In [57]: name='rakesh'  
  
def greet(name='Alex'):  
    print(f"hello {name}")  
  
name='Sanjay'  
greet()
```

```
hello Alex
```

```
In [58]: name='rakesh'  
  
def greet():  
    print(f"hello {name}")  
  
name='Sanjay'  
greet()
```

```
hello Sanjay
```

```
In [59]: name='rakesh'  
  
def greet():  
    print(f"hello {name}")  
  
greet()
```

```
hello rakesh
```

```
In [62]: a=10
def val():
    b=30
    c=60
    a=b+c
    return(a)

a=val()
```

```
In [63]: a
```

```
Out[63]: 90
```

```
In [ ]: # Q) How can I take the value 90
        # Out side the functions

        #Q) How can I take the value 90
        # Out side the functions with out using return
```

```
In [73]: a=10
def val():
    global c1
    b1=30
    c1=60
    a=b1+c1

val()
```

```
In [74]: c1
```

```
Out[74]: 60
```

```
In [75]: a=10
def val():
    global b1,c1
    b1=300
    c1=600
    a=b1+c1

val()
```

```
In [77]: b1,c1
```

```
Out[77]: (300, 600)
```

Functions in Functions

```
In [ ]: # We want to call one function inside other function
```

```
In [79]: def greet1():  
         print("hello")  
  
         def greet2():  
             print("how are you")  
  
         greet1()  
         greet2()
```

```
hello  
how are you
```

```
In [81]: def greet1():  
         print("hello")  
         print("hey")  
  
         def greet2():  
             greet1() # calling greet1  
             print("how are you")  
  
         greet2()  
         # It will call greet1()
```

```
hello  
hey  
how are you
```

```
In [83]: def greet1():  
         print("hello")  
         print("hey")  
         greet2()  
  
         def greet2():  
             print("how are you")  
  
         greet1()
```

```
hello  
hey  
how are you
```

```
In [84]: def greet1():  
          print("hello")  
          print("hey")  
          greet2()  
  
          def greet2():  
              greet1() # calling greet1  
              print("how are you")  
  
          greet2()
```

```
hello  
hey  
hello  
hey  
hello  
hey  
hello  
hey  
hello  
hey  
hello  
hey  
hello  
hey  
hello  
hey  
hello  
hey  
hello  
.
```

```
In [ ]: # You need to create 4 individual functions  
        # every function will provide two arguments  
        # add  
        # sub  
        # mul  
        # div  
  
        # write a print statement to communicate user  
        # if you enter 1 then add function will run  
        # if you enter 2 then sub function will run  
        # if you enter 3 then mul function will run  
        # if you enter 4 then div function will run  
  
        # operation=eval(input("enter which operation you want to perform"))  
        # if operation==1:  
        #     add()  
  
        # elif operation==2:  
        #     sub()
```

```
In [ ]:
```

```
In [ ]:
```

In []:

In []:

In []: